1. (1 Punkt) Was wird ausgegeben?

```
a = [5]
b = a
a.append(8)
print(a, b)
```

Lösung:

```
[5, 8] [5, 8]
```

2. (1 Punkt) Was wird ausgegeben?

```
a = [7]
b = a.copy()
b.append(8)
print(a, b)
```

Lösung:

```
[7] [7, 8]
```

3. (1 Punkt) Was wird ausgegeben?

```
c = [12]
d = c
d.append(8)
c.append(4)
print(c, d)
```

Lösung:

```
[12, 8, 4] [12, 8, 4]
```

4. (1 Punkt) Was wird ausgegeben?

```
a = [4]
b = a
a = a + [4]
print(a, b)
```

Lösung:

```
[4, 4] [4]
```

5. (2 Punkte) Was wird ausgegeben?

```
a = [1]
b = a
a += [2]
a = a + [3]
b += [4]
print(a, b)
```

Lösung:

```
[1, 2, 3] [1, 2, 4]
```

`a += [2]` verändert die gemeinsame Liste. `a = a + [3]` erzeugt eine neue Liste für `a`, `b += [4]` verändert die alte Liste, auf die nur noch `b` zeigt.

6. (2 Punkte) Was wird ausgegeben?

```
a = [3, 1, 2]
b = a
c = sorted(a)
a.append(0)
d = a.sort()
print(a, b, c, d)
```

Lösung:

```
[0, 1, 2, 3] [0, 1, 2, 3] [1, 2, 3] None
```

`sorted` erzeugt eine neue Liste, `sort` verändert die Liste und gibt `None` zurück.

7. (2 Punkte) Was wird ausgegeben?

```
a = [4]
b = [6]
c = (a,b)
d = c
d[1].append(1)
print(d, c)
```

Lösung:

```
([4], [6, 1]) ([4], [6, 1])
```

8. (2 Punkte) Tupels sind unveränderbar. Kann sich die Ausgabe eines Tupels t ändern, ohne dass t ein neuer Wert zugewiesen wird? Begründe deine Antwort.

Lösung: Ja, wenn das Tupel z.B. eine Liste enthält. In diesem Fall kann sich der Inhalt der Liste ändern, ohne dass das Tupel selbst verändert wird. Beispiel:

```
a = [3]
t = (a,0)
print(t)
a.append(8)
print(t)

Ausgabe:
([3], 0)
([3, 8], 0)
```

9. (2 Punkte) Was wird ausgegeben?

```
d = {'a': 1}
e = d
f = d.copy()
e['b'] = 2
f['a'] = 5
print(d, e, f)
```

Lösung:

```
{'a': 1, 'b': 2} {'a': 1, 'b': 2} {'a': 5}
```

10. (1 Punkt) Was wird ausgegeben?

```
def doit(x):
    x.append(7)
    return x

a = [4]
b = doit(a)
print(a, b)
```

Lösung:

```
[4, 7] [4, 7]
```

11. (1 Punkt) Was wird ausgegeben?

```
def doit(x):
    x = []
    x.append(7)
    return x

a = [4]
b = doit(a)
print(a, b)
```

Lösung:

```
[4] [7]
```

12. (2 Punkte) Was wird ausgegeben?

```
def doit(x):
    x.append(12)
    a = [7]
    return x

a = [4]
b = doit(a)
print(a, b)
```

Lösung:

```
[4, 12] [4, 12]
```

13. (2 Punkte) Was wird ausgegeben?

```
def doit(x):
    a = x.copy()
    a.append(7)
    return x

a = [4]
b = doit(a)
print(a, b)
```

Lösung:

```
[4] [4]
```

14. (4 Punkte) (a) (2 Punkte) Schreibe eine Funktion `ohne_negative(a)`, die eine neue Liste mit allen nicht negativen Zahlen der Liste `a` zurückgibt. Die Liste `a` soll dabei nicht verändert werden.

    (b) (1 Punkt) Schreibe eine Funktion `negative_auf_null(a)`, die in der Liste `a` alle negativen Zahlen durch 0 ersetzt. Die Funktion gibt nichts zurück.

    (c) (1 Punkt) Was gibt das folgende Programm aus?

```
x = [3, -1, 2]
print(ohne_negative(x), x)
y = negative_auf_null(x)
print(x, y)
```

Lösung:

```
# a)
def ohne_negative(a):
    ergebnis = []
    for x in a:
        if x >= 0:
            ergebnis.append(x)
    return ergebnis

# b)
def negative_auf_null(a):
    for i in range(len(a)):
        if a[i] < 0:
            a[i] = 0

c)  [3, 2] [3, -1, 2]
    [3, 0, 2] None
```

15. (2 Punkte) Was wird ausgegeben?

```
a = [1]
b = [3]
c = b.copy()
d = [a,c,b]
b.append(4)
print(a,b,c,d)
```

Lösung:

```
[1] [3, 4] [3] [[1], [3], [3, 4]]
```

16. (3 Punkte) Was wird ausgegeben?

```
from copy import deepcopy

a = [1]
b = [a, [2]]
c = b.copy()
d = deepcopy(b)
a.append(3)
c.append([4])
print(b)
print(c)
print(d)
```

Lösung:

```
[[1, 3], [2]]          b enthält einen Zeiger auf a
[[1, 3], [2], [4]]     flache Kopie: neue Liste, aber derselbe Zeiger auf a
[[1], [2]]             tiefe Kopie: auch a wurde kopiert
```

17. (3 Punkte) (a) (1 Punkt) Was wird ausgegeben?

```
feld = [[0] * 2] * 3
feld[1][0] = 5
print(feld)
```

(b) (2 Punkte) Schreibe Anweisungen, die ein Feld mit 3 Zeilen und 2 Spalten voller Nullen so erzeugen, dass `feld[1][0] = 5` nur die mittlere Zeile verändert.

Lösung: a) `[[5, 0], [5, 0], [5, 0]]`, alle drei Zeilen sind dieselbe Liste.

```
# b)
feld = []
for i in range(3):
    feld.append([0] * 2)
```